

SDN Load Balancer

Project Report

CS 3001 — Computer Networks

Muhammad Umer Fazal

FAST School of Computing

National University of Computer & Emerging Sciences

1. Motivation

Modern networks are experiencing unprecedented growth in traffic volumes, driven by the rapid expansion of cloud computing, video streaming, e-commerce platforms, and IoT devices. Traditional hardware-based load balancers, while effective, present significant limitations: they are prohibitively expensive for small and medium enterprises, extremely difficult to configure and manage, and fundamentally inflexible in adapting to the dynamic demands of contemporary applications.

The emergence of Software Defined Networking (SDN) has introduced a revolutionary paradigm that decouples the control plane from the data forwarding plane, enabling centralized, programmable network management. This project is motivated by the compelling opportunity to leverage SDN principles to design and implement a dynamic, intelligent load balancer that is not only cost-effective but also highly configurable and adaptable. By utilizing the POX SDN controller and the Mininet network emulator, this project demonstrates how software-defined approaches can replace expensive hardware solutions with flexible, programmable alternatives that can be deployed, modified, and scaled with minimal effort.

Furthermore, this project serves as a practical academic exploration of core networking concepts including OpenFlow protocol operation, flow table management, network address translation, ARP-based health monitoring, and load distribution algorithms — all of which are fundamental to modern network engineering and directly relevant to real-world industry applications.

2. Overview

This project designs and implements an SDN-based dynamic load balancer. A centralized POX controller manages traffic distribution among multiple backend HTTP servers within

a Mininet-emulated network environment. The system intercepts client TCP connections directed at a Virtual IP (VIP) and intelligently redirects them to appropriate backend servers using programmable load balancing algorithms.

2.1 Significance of the Project

Load balancing is a cornerstone of modern network infrastructure, enabling high availability, fault tolerance, and efficient resource utilization. The significance of this project spans several dimensions:

- **Practical Relevance:** Load balancing is used in virtually every large-scale web service. Understanding its implementation at the network level provides deep insights into how the internet functions at scale.
- **Cost-Effectiveness:** Hardware load balancers can cost tens of thousands of dollars. An SDN-based software implementation achieves similar functionality at a fraction of the cost.
- **Academic Value:** The project integrates concepts from multiple areas of networking including OpenFlow, packet processing, NAT, ARP probing, flow rule management, and algorithm design.
- **Impact:** A working SDN load balancer demonstrates that programmable networks can deliver enterprise-grade traffic management capabilities through pure software.

2.2 Description of the Project

The system comprises a single OpenFlow-enabled switch (Open vSwitch), multiple server nodes running HTTP web services, and numerous client nodes. A Virtual IP (VIP) address serves as the single entry point for all client requests. When a client initiates a TCP connection to the VIP, the switch forwards the first packet to the POX controller. The controller selects an appropriate backend server, rewrites IP and MAC addresses, installs bidirectional OpenFlow flow rules, and forwards the packet. Subsequent packets of the same connection are handled directly by the switch without controller involvement.

The project scope includes:

- A Mininet topology with 1 switch, 4 servers (10.0.0.101–104), and 9 clients (10.0.0.11–19).
- A Virtual IP address of 10.0.0.100 representing the load-balanced service.
- Two load balancing algorithms: Round Robin and Least Connection.
- ARP-based server health monitoring with automatic failover.
- Session persistence ensuring all packets of a connection reach the same server.
- A simple DNS server resolving domain names to the VIP address.

2.3 Background of the Project

- OpenFlow Protocol: McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," ACM SIGCOMM CCR, 2008.
- Software Defined Networking: Feamster, Rexford & Zegura, "The Road to SDN," ACM SIGCOMM CCR.
- POX Controller Documentation: <https://noxrepo.github.io/pox-doc/html/>
- Mininet Documentation: <http://mininet.org/documentation/>
- Open Networking Foundation: <https://opennetworking.org>

2.4 Project Category

This project falls under both Research-Based and Product-Based categories. It investigates the feasibility and effectiveness of SDN-driven load balancing as an alternative to hardware-based solutions, while also delivering a fully functional, demonstrable system capable of handling real HTTP traffic.

3. Features / Scope / Modules

1. Dynamic Load Balancing

The system dynamically distributes incoming TCP connections across multiple backend servers in real time. The POX controller intercepts new connection requests and makes forwarding decisions transparently to the client, which only ever sees the VIP address.

2. Round Robin Algorithm

Distributes requests sequentially across all live servers in a circular order using a modulo operation. Each server receives an equal share of connections, ideal for homogeneous workloads.

3. Least Connection Algorithm

Always routes new requests to the server with the fewest active connections. Connection counts are incremented on new flows and decremented on expiry, making this algorithm more adaptive for heterogeneous workloads.

4. ARP-Based Server Health Monitoring

The controller sends ARP requests to all servers every 5 seconds. Servers failing to reply within 3 seconds are removed from the live pool and traffic is automatically redirected to remaining healthy servers.

5. OpenFlow Flow Rule Installation

The controller installs bidirectional flow rules in the switch for each new connection, enabling direct packet forwarding without controller involvement for subsequent packets, improving throughput dramatically.

6. Session Persistence (Sticky Sessions)

All packets of a connection are consistently forwarded to the same backend server, identified by the 5-tuple (src IP, dst IP, src port, dst port, protocol). Mappings are stored for up to 5 minutes.

7. DNS Resolution for Virtual Service

A lightweight UDP DNS server runs as a background thread, listening on port 53 and resolving domain queries to the VIP address (10.0.0.100), enabling domain-name-based access to the service.

8. Network Topology Simulation via Mininet

The complete network environment including 1 switch, 4 servers, and 9 clients is emulated using Mininet with Open vSwitch, enabling comprehensive testing of all load balancing behavior.

9. HTTP Service Simulation

Each backend server runs Python's built-in HTTP server on port 80, enabling realistic HTTP request/response traffic testing using curl or wget from any client.

4. Project Planning

The project was executed over a five-week timeline with responsibilities divided between both team members:

Task	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5
SDN & OpenFlow Research	■				
Install & Configure Mininet	■				
Install & Configure POX	■				
Implement Mininet Topology		■			
Verify Connectivity (ping tests)		■			
Implement Packet Handling in POX		■			
Implement Round Robin Algorithm			■		
Configure HTTP Web Servers			■		
Implement Least Connection Algorithm			■		
Implement Connection Tracking			■		
Install OpenFlow Flow Rules				■	
Implement Session Persistence				■	
ARP-Based Health Monitoring				■	
Failover Logic for Dead Servers				■	
Implement DNS Server					■
Integration Testing					■

Task	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5
Performance Analysis & Debugging					■

5. Project Feasibility

5.1 Technical Feasibility

The project is fully technically feasible using available open-source tools. All components are mature and well-documented:

- Mininet (v2.3+) provides reliable network emulation on any Linux system.
- POX is a production-tested Python SDN controller used in numerous academic projects.
- Open vSwitch provides full OpenFlow 1.0 support.
- Python 3 provides all necessary standard libraries (socket, threading, time).

Technical risks are minimal: Python 3.13 compatibility warnings from POX do not affect functionality, and Mininet can be installed on Kali Linux via apt despite the install script not officially supporting it.

5.2 Economic Feasibility

The project has negligible economic cost as all components are free and open-source (Mininet: BSD; POX: Apache 2.0; Open vSwitch: Apache 2.0; Python: PSF). Any standard PC or laptop running Linux is sufficient — no specialized hardware is required. This contrasts sharply with hardware load balancers costing \$10,000–\$50,000 or more.

5.3 Schedule Feasibility

The five-week timeline is adequate for the defined scope. The modular architecture allowed parallel development by both team members. All major features were

implemented and tested within the allocated timeframe, with the final week reserved for integration testing and optimization.

6. Hardware and Software Requirements

6.1 Hardware Requirements

Component	Minimum	Recommended
CPU	Dual-core 2.0 GHz	Quad-core 2.5 GHz+
RAM	4 GB	8 GB or more
Storage	20 GB free	50 GB SSD
OS	Linux (any)	Ubuntu 20.04 / Kali 2026

6.2 Software Requirements

Software	Version	Purpose	License
Linux OS	Ubuntu 20.04+ / Kali 2026	Host operating system	Open Source
Python 3	3.8+	Controller scripting	PSF (Free)
Mininet	2.3.1+	Network emulation	BSD (Free)
Open vSwitch	2.13+	Virtual OpenFlow switch	Apache 2.0
POX Controller	0.7.0 (gar)	SDN controller framework	Apache 2.0
Python http.server	Built-in	Simulate web servers	PSF (Free)
curl / wget	Any	Generate HTTP traffic	Open Source
Git	2.x+	Clone POX repository	GPL (Free)

7. Diagrammatic Representation of the Overall System

The following diagrams illustrate the complete system architecture and internal logic of all components:

Figure 1: Overall System Architecture

Shows the relationship between clients, the OpenFlow switch, POX controller, VIP, DNS server, and backend servers including the data flow and ARP probe mechanism.

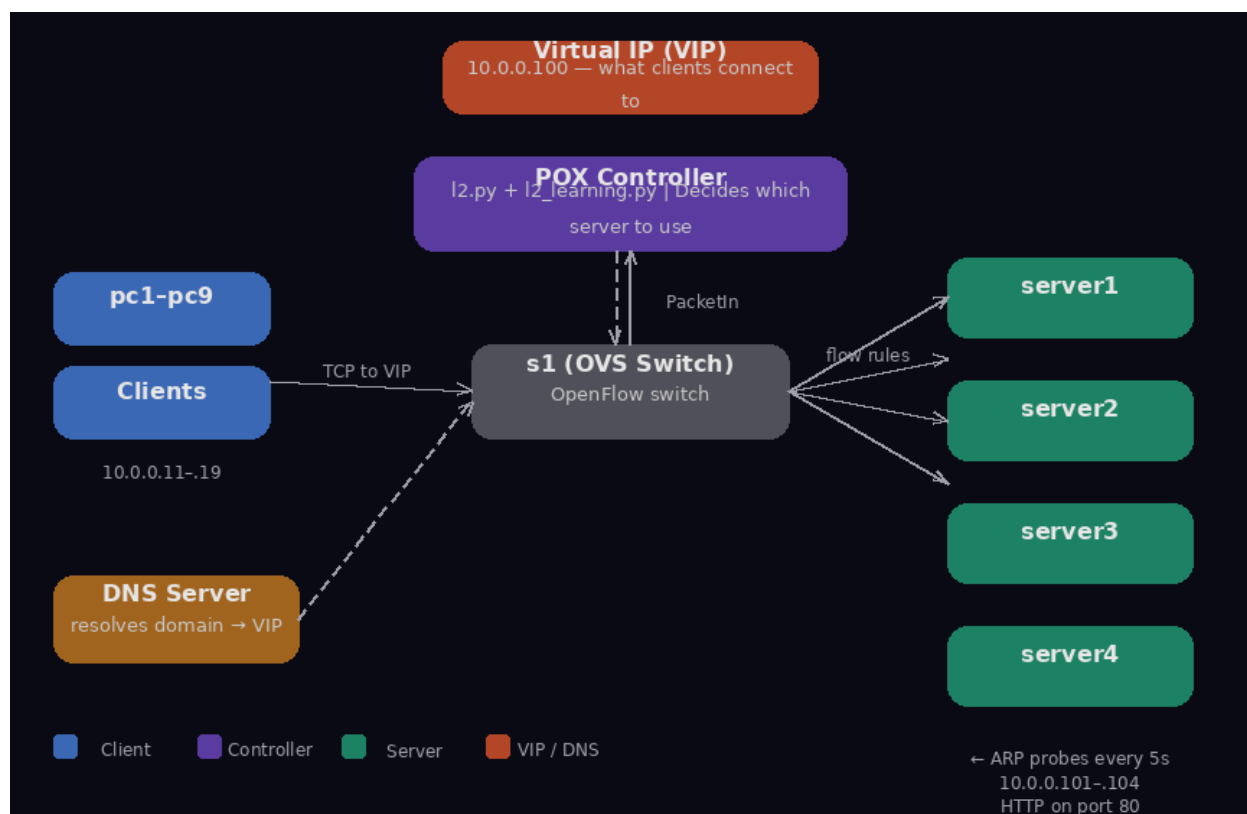


Figure 1: SDN Load Balancer — Overall System Architecture

Figure 2: L2 Learning Switch Logic (l2_learning.py)

Flowchart showing how the learning switch builds its MAC-to-port table and decides whether to flood, install a flow rule, or drop packets.

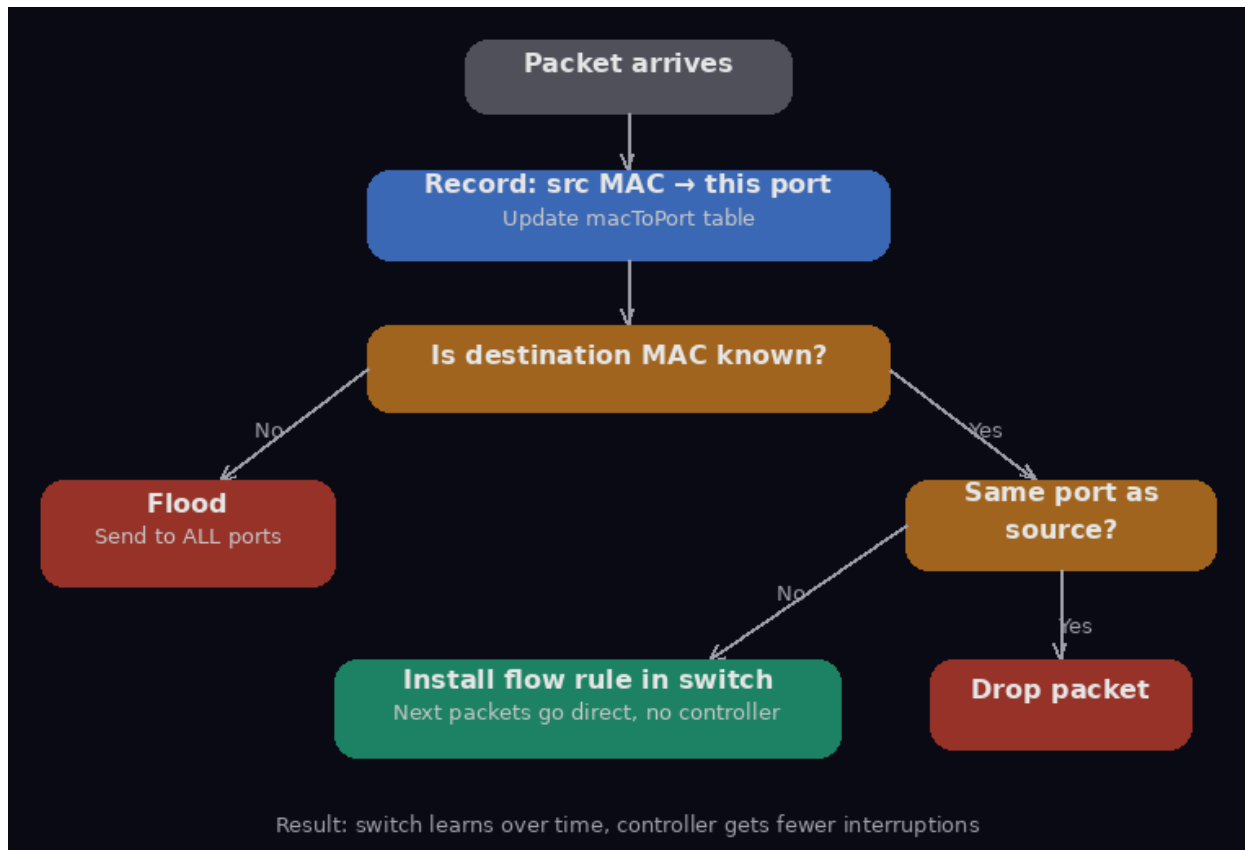


Figure 2: L2 Learning Switch — Packet Handling Flowchart

Figure 3: I2.py Internal Components

Component diagram showing the four major classes and data structures inside I2.py: DNSServer, MemoryEntry, launch(), and the iplb class with all its methods and dictionaries.

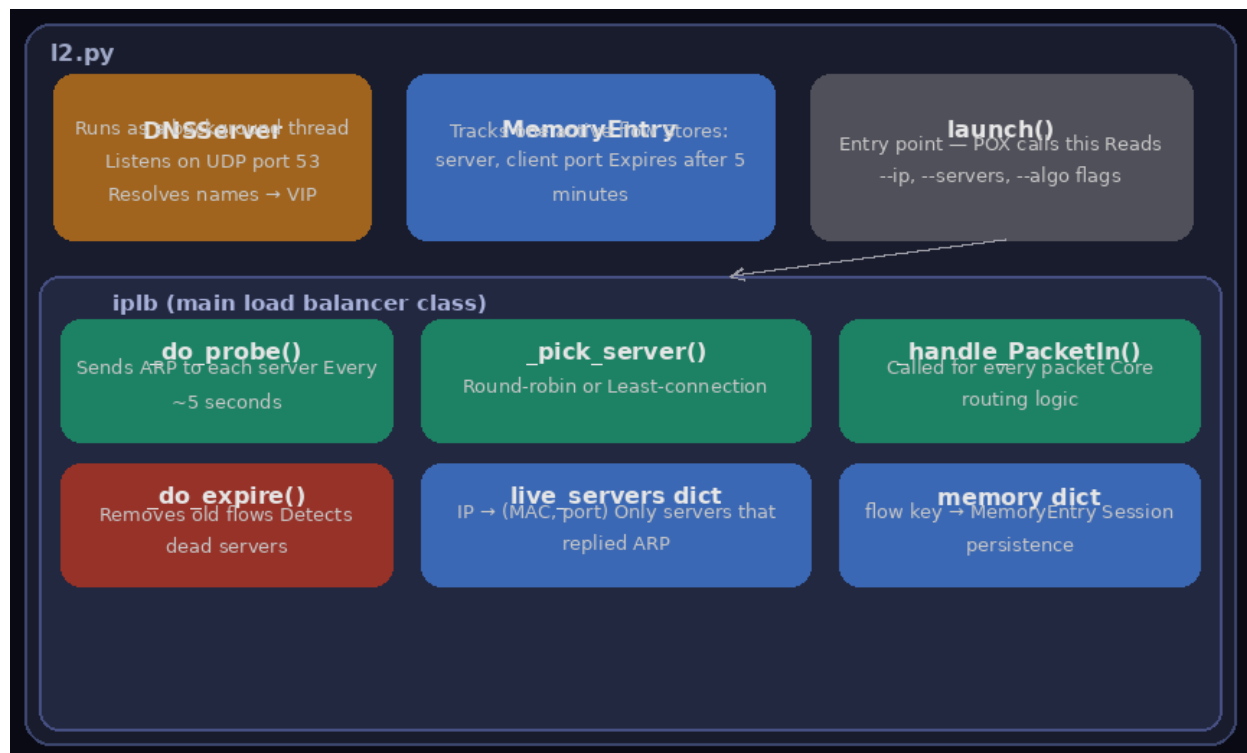


Figure 3: I2.py — Internal Class and Component Structure

Figure 4: `_handle_PacketIn()` Logic (I2.py)

Detailed flowchart of the core routing logic that runs every time the controller receives a packet, including TCP/ARP handling, server selection, IP rewriting, and flow rule installation.

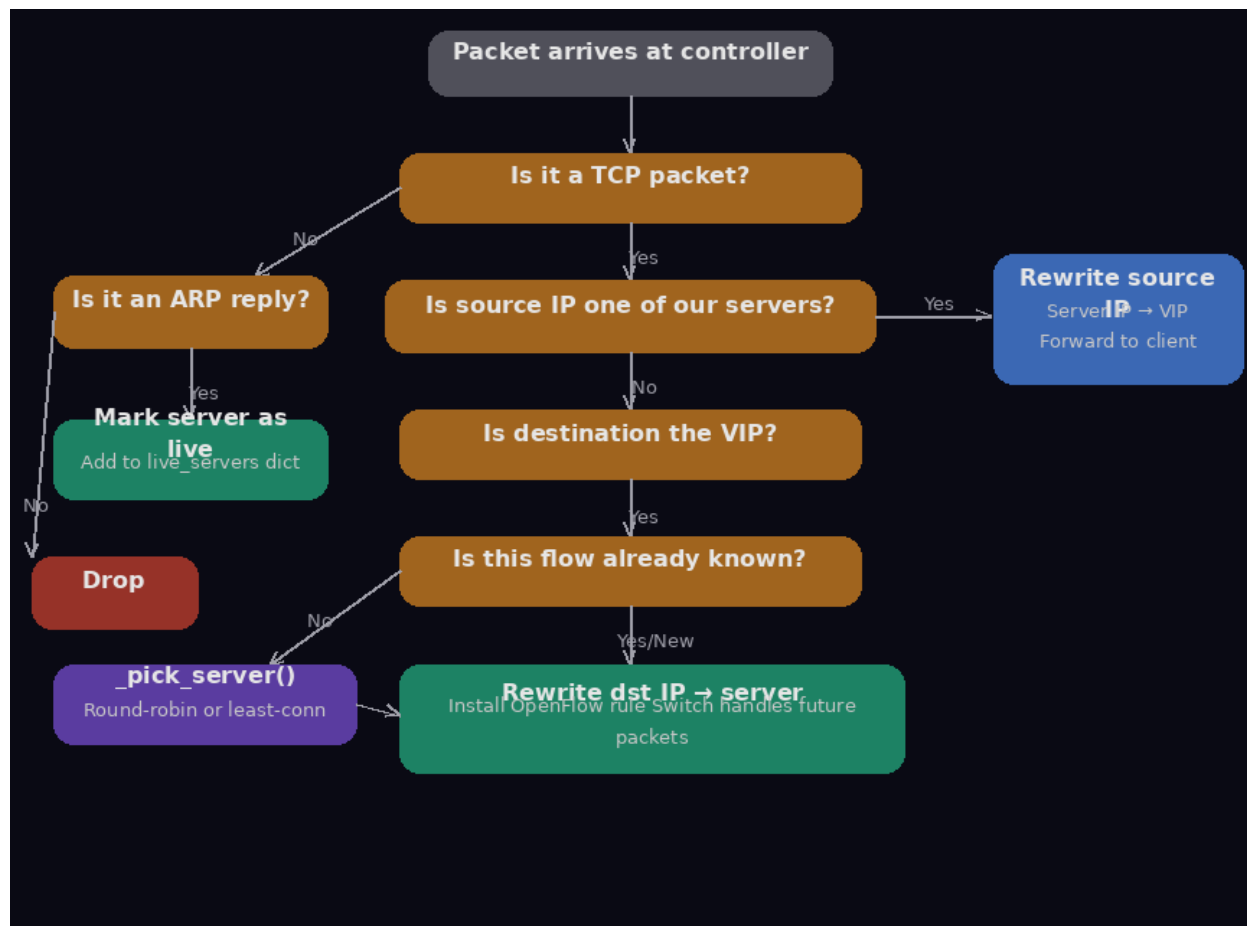


Figure 4: `_handle_PacketIn()` — Core Load Balancer Flowchart

References

1. McKeown, N., et al. "OpenFlow: Enabling Innovation in Campus Networks." ACM SIGCOMM CCR, 2008.
2. Open Networking Foundation. <https://opennetworking.org>
3. POX Controller Documentation. <https://noxrepo.github.io/pox-doc/html/>
4. Mininet Documentation. <http://mininet.org/documentation/>
5. Feamster, N., Rexford, J., & Zegura, E. "The Road to SDN." ACM SIGCOMM CCR.